GMI German-Malaysian Institute

DIPLOMA PROGRAMME

MINI PROJECT

Academic Period : July 2025

Program	:	**DIPLOMA OF SOFTWARE ENGINEERING**
Course	:	**DATA STRUCTURE AND ALGORITHMS**
Code	:	**CIT 0663**
Duration	:	**4 HOURS**
Total Marks	:	**100**

Title : **QUEUE METHOD**

Student's Name :
(Individual / Group)

3.NUR FATIN FASISHAH BINTI M.RUSDEI@RUSDI

TTO's Name :

Semester/Trade/Group :

Date :

Assessment Specification Table (SWE)

Assignment

No	Assessment Criteria	Full Marks	Score (if applicable)	Student's Marks	CLO	Level of Difficulty	DK. DP, NA (ETAC Programme Only)	Comments	GMI PLO: PLO SET (10 PLO) Previous Intake Jan 2023 and below	GMI PLO: PLO SWE (11 PLO) Intake July 2023
Part 1: Report	Introduction	5			1	C3			1	1
	Pseudocode & Flowchart	20			1	C3			1	1
	Program functionality in accordance with an output	30			1	C3			1	1
	Conclusion	5			1	C3			1	1
Part 2: Others	Documentation Format	10			3	A3			8	5
	Plagiarism	5			3	A3			8	5
	Presentation	15			3	A3			8	5
Total Full Marks		**90**	**Total Student's Marks**							

SUMMARY

Note:
FM - Full Marks,
StM - Student's Mark

Total

CLO	Raw Marks FM	Raw Marks StM	Percentage FM	Percentage StM	PLO (PLO SET)	Raw Marks FM	Raw Marks StM	Percentage FM	Percentage StM	PLO (PLO SWE)	Raw Marks FM	Raw Marks StM	Percentage FM	Percentage StM
CLO 1	60		20.0		PLO 1	60		20.0		PLO 1	60		20.0	
CLO 2	0		0.0		PLO 4	0		0.0		PLO 3	0		0.0	
CLO 3	30		10.0		PLO 8	30		10.0		PLO 5	30		10.0	
Total Marks	**90**		**30.0**											

Signature

( )

Problem Statement

You are currently working as a Programmer for a new restaurant, "The Flavorful Fork." The client, the restaurant owner, has asked you to create a program to manage cooking orders. The program should use C language as its primary programming language and function as a cooking order queue system. The system should allow kitchen staff to add new orders, remove completed orders, view the current list of orders, and see the total number of orders in the queue. Here is a sample menu that you can use as your reference:

*********** WELCOME TO**********

********** THE FLAVORFUL FORK **********

A: Add Order

B: Remove Order

C: List of Orders

D: Total Number of Orders

Q: Quit or Logout

****

Enter your option:

You are required to write a program that has 5 menu options for users to choose from. You can program it using your own creativity, and marks will be awarded for that. Here are the details for each option:

OPTION	**DESCRIPTION**
Add Order	Register a new food order (e.g. “Spaghetti Carbonara”, “Chicken Salad”).
Remove Order	Remove a completed order from the queue.
List of Orders	List all the food orders that are currently in queue.
Total Number of Orders	Display the total number of orders that are in queue.
Quit or Logout	
When this option is chosen, it will clear terminal and print the main menu.	

Your Deliverables:

Prepare a **SOFTCOPY REPORT** and **SLIDE PRESENTATION** (including a demonstration of the completed program). It must include:

1. **Introduction**
2. **Pseudocode & Flowchart**
3. **Program functionality in accordance with an output**
4. **Conclusion**: Summarize the advantages and potential disadvantages of your chosen method, and its suitability for GMI's context.
5. **Documentation format**

   Report [format]

   - Font type: Arial / Times New Roman,
   - Line Spacing: 1.5 / 2.0
   - Paragraph format: Justify
   - Presentation and submission of report by or before:

CRITERIA	MARKS ALLOCATED	MARKS (M)		WEIGHTED MARKS OBTAINED
				M/5 x 5% =
Introduction	5%	**1**	Not able to discuss the given task	
		2	Minimal ability to discuss the given task	
		3	Some ability to discuss the given task	
		4	Able to discuss the given task	
		5	Able to discuss with good illustration	
				M/5 x 20% =
Pseudocode & Flowcharts	20%	**1**	Correct use of flow chart symbols & conventions	
		2	Correct placement of logical flow	
		3	Correct selection of instructions	
		4	Correct placement of instruction iterations	
		5	Correct use of conventions, symbols, instructions and iterations	
				M/5 x 30% =
Program functionality in accordance with an output	30%	**1**	There is no correlation of program functionality with no output related	
		2	There is little correlation of program functionality to an output	
		3	There is partial correlation of program functionality to an output	
		4	There is sufficient correlation of program functionality to an output	
		5	There is complete correlation of program functionality to an output	
				M/5 x 5% =
Conclusion	5%	**1**	Conclusion is not available	
		2	No sensible conclusion	
		3	A conclusion is drawn, but not supported by evidence	

		4	A conclusion is drawn, but with some support of evidence	
		5	A conclusion is drawn, with complete support of evidence	
				M/5 x 10% =
Documentation Format	10%	**1**	The document was incorrectly formatted	
		2	The document was partially formatted	
		3	The document was adequately formatted	
		4	The document was well formatted	
		5	The document was very well formatted	
				M/5 x 5% =
Plagiarism	5%	**1**	Plagiarism	
		2	Inadequate citation	
		3	Minimal citation	
		4	Adequate Citation	
		5	Original Work and proper citation	
				M/5 x 15% =
Presentation	15%	**1**	No eye contact with audience and audience not pay attention.	
		2	Minimal eye contact by more than one member focusing on small part of audience	
		3	Members focused on only part of audience and the audience was distracted	
		4	Most member spoke to majority of audience; steady eye contact	
		5	Regular/ constant eye contact. The audience was engaged and audience pay more attention.	
TOTAL	**90**			**/ 90**
				/30

TABLE OF CONTENTS

NUM	CONTENTS	PAGES
1.0	Introduction	7
2.0	2.1 Pseudocode 2.2 Flowchart	8 9
3.0	3.1 C coding 3.2 Queue Representation in the Program 3.3 Output	10 - 13 14 - 17 18 - 22
4.0	Conclusion	23
5.0	References	24

1.0 Introduction

In modern restaurant settings, managing food orders is essential for smooth kitchen operations and customer satisfaction. With rising customer demand and quick service expectations, restaurants need systems that can organize and process orders efficiently and on time. This mini project focuses on creating a Cooking Order Queue System using the C programming language and the Queue data structure. A queue follows the FIFO (First-In, First-Out) principle, which means the first order received will be the first to be prepared and served. This approach mirrors real-life workflows in restaurant kitchens, where orders must be handled fairly based on the time they were placed.

The proposed system, designed for "The Flavorful Fork" restaurant, enables kitchen staff to add new food orders, remove completed orders, list all current orders, and show the total number of pending orders. By using queue operations such as enqueue and dequeue, this project simulates real restaurant order management. It demonstrates how data structures can solve practical problems by improving organization, reducing errors, and ensuring a smooth process.

This mini project enhances understanding of basic data structures and improves skills in designing algorithms, program logic, and modular programming in C. It also promotes problem-solving, logical thinking, and the ability to turn real-life situations into functional computer programs. Through this project, students gain hands-on experience in using abstract data structures to create efficient systems. The system developed is simple, efficient, and appropriate for academic learning within GMI's Data Structure and Algorithms course, while also showing how such systems can be applied in real restaurant operations.

2.0 Pseudocode and Flowchart

2.1 Pseudocode

```
START
      Initialize Queue as empty

DISPLAY_MAIN_MENU :
    Display:
     A.  Add Order
     B.  Remove Order
     C.  List All Orders
     D.  Total Number of Orders
     E.  Quit

   Input option (A/B/C/D/E)

   IF option == 'A' THEN
      Input orderName
      Add orderName to Queue
      Go to DISPLAY_MAIN_MENU

   ELSE IF option == 'B' THEN
   IF Queue is empty THEN
       Display " No orders to remove"
   ELSE
       Remove first order from Queue ( Dequeue)
       Display " First order removed"
   ENDIF
   Go to DISPLAY_MAIN_MENU

  ELSE IF option == 'C' THEN
     IF Queue is empty THEN
        Display "No orders
     ELSE
         Display all orders in Queue
    ENDIF
    Go to DISPLAY_MAIN_MENU

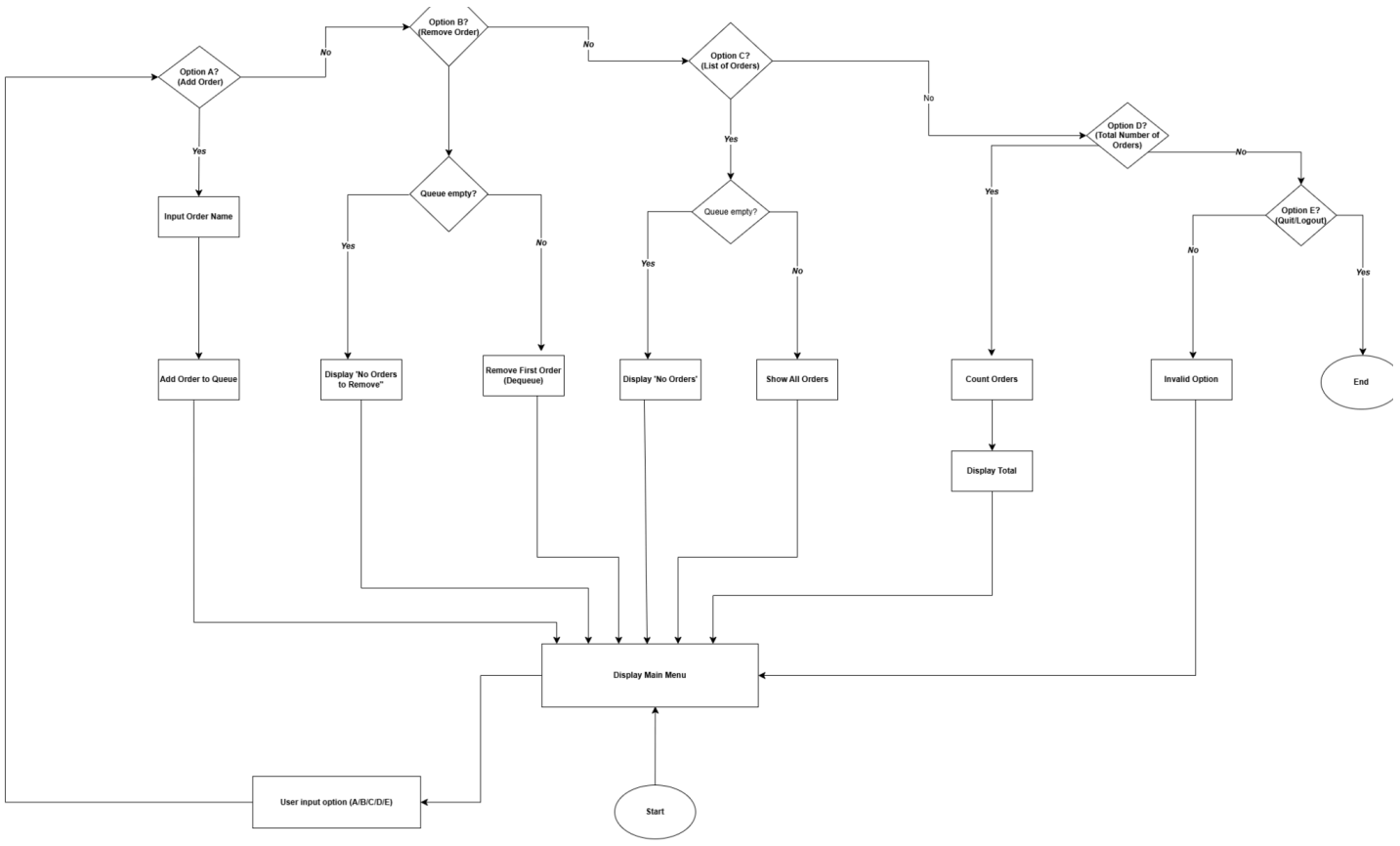
  ELSE IF option == 'D' THEN
       Count number of items in Queue
       Display total
       Go to DISPLAY_MAIN_MENU

 ELSE IF option == 'E' THEN
      Display " Exiting program…"
      END PROGRAM

 ELSE
    Display " Invalid option"
    Go to DISPLAY_MAIN_MENU
```

```
    ENDIF
END
```

2.2 Flowchart



3.0 Program functionality in accordance with an output

3.1 C Program Coding

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_SIZE 100
#define MAX_ORDER_LENGTH 100

char queue[MAX_SIZE][MAX_ORDER_LENGTH];
int front = -1;
int rear = -1;

void displayMainMenu();
void addOrder();
void removeOrder();
void listOrders();
void totalOrders();
int isEmpty();
int isFull();
void clearScreen();
void pauseScreen();

int main() {
   char option;

   printf("\n=========================================\n");
   printf("  RESTAURANT ORDER MANAGEMENT SYSTEM\n");
   printf("=========================================\n");
   pauseScreen();

   // Main program loop - matches flowchart
   while(1) {
      displayMainMenu();

      printf("Enter your option: ");
      scanf(" %c", &option);
      getchar(); // Clear newline from buffer

      option = toupper(option);

      if(option == 'A') {
         addOrder();
      }
      else if(option == 'B') {
         removeOrder();
      }
      else if(option == 'C') {
```

```
            listOrders();
        }
        else if(option == 'D') {
            totalOrders();
        }
        else if(option == 'E') {
            clearScreen();
            printf("\n=========================================\n");
            printf("       Exiting program...\n");
            printf("   Thank you for using the system!\n");
            printf("=========================================\n\n");
            exit(0);
        }
        else {
            printf("\nInvalid option. Please try again.\n");
            pauseScreen();
        }
    }

    return 0;
}

void displayMainMenu() {
    clearScreen();
    printf("\n*****************************************\n");
    printf("***************** WELCOME TO ****************\n");
    printf("***************** THE FLAVORFUL FORK ****************\n");
    printf("*****************************************\n");
    printf("A: Add Order\n");
    printf("B: Remove Order\n");
    printf("C: List of Orders\n");
    printf("D: Total Number of Orders\n");
    printf("E: Quit or Logout\n");
    printf("*****************************************\n\n");
}

int isEmpty() {
    return (front == -1 || front > rear);
}

int isFull() {
    return (rear == MAX_SIZE - 1);
}

void addOrder() {
    clearScreen();
    printf("\n=========================================\n");
    printf("           ADD ORDER                  \n");
    printf("=========================================\n\n");

    char orderName[MAX_ORDER_LENGTH];
```

```
    printf("Enter order name: ");
    fgets(orderName, MAX_ORDER_LENGTH, stdin);
    orderName[strcspn(orderName, "\n")] = 0;

    if(strlen(orderName) == 0) {
        printf("\nOrder name cannot be empty!\n");
        pauseScreen();
        return;
    }

    if(isFull()) {
        printf("\nQueue is full! Cannot add more orders.\n");
        pauseScreen();
        return;
    }

    if(front == -1) {
        front = 0;
    }
    rear++;
    strcpy(queue[rear], orderName);

    printf("\nOrder added successfully!\n");
    printf("Order: %s\n", orderName);
    printf("Position in queue: %d\n", rear - front + 1);
    pauseScreen();
}

void removeOrder() {
    clearScreen();
    printf("\n==========================================\n");
    printf("         REMOVE ORDER                    \n");
    printf("==========================================\n\n");

    if(isEmpty()) {
        printf("No orders to remove.\n");
        printf("Queue is empty!\n");
        pauseScreen();
        return;
    }

    printf("Removing order: %s\n", queue[front]);
    front++;

    if(front > rear) {
        front = -1;
        rear = -1;
        printf("\nFirst order removed successfully!\n");
        printf("Queue is now empty.\n");
    }
```

```
    else {
        printf("\nFirst order removed successfully!\n");
        printf("Orders remaining: %d\n", rear - front + 1);
    }

    pauseScreen();
}

void listOrders() {
    clearScreen();
    printf("\n=============================================\n");
    printf("        LIST OF ORDERS                \n");
    printf("=============================================\n\n");

    if(isEmpty()) {
        printf("No orders in queue.\n");
        printf("Queue is empty!\n");
        pauseScreen();
        return;
    }

    printf("Current Orders in Queue:\n");
    printf("-----------------------------------------\n");
    printf("%-5s %-35s\n", "No.", "Order Name");
    printf("-----------------------------------------\n");

    for(int i = front; i <= rear; i++) {
        printf("%-5d %-35s\n", i - front + 1, queue[i]);
    }

    printf("-----------------------------------------\n");
    printf("Total: %d order(s) in queue\n", rear - front + 1);
    pauseScreen();
}

void totalOrders() {
    clearScreen();
    printf("\n=============================================\n");
    printf("     TOTAL NUMBER OF ORDERS           \n");
    printf("=============================================\n\n");

    if(isEmpty()) {
        printf("Total orders: 0\n");
        printf("Queue is currently empty.\n");
    }
    else {
        int total = rear - front + 1;
        printf("Total orders in queue: %d\n", total);
        printf("\nQueue Status:\n");
        printf("First order (Front): %s\n", queue[front]);
        printf("Last order (Rear):   %s\n", queue[rear]);
```

```
    }

    pauseScreen();
}

void clearScreen() {
    #ifdef _WIN32
        system("cls");
    #else
        system("clear");
    #endif
}

void pauseScreen() {
    printf("\nPress Enter to continue...");
    getchar();
}
```

3.2 Queue Representation in the Program

The queue in this program is implemented using a linear array, which serves as the storage for all customer food orders. The system follows the FIFO (First-In, First-Out) principle, meaning that the earliest order added will always be the first one to be processed and removed. This behavior is essential for restaurant operations, where fairness and sequence are important to avoid serving later customers before earlier ones. The queue helps maintain this strict order by ensuring that each incoming order is placed at the rear and removed from the front.

```
char queue[MAX_SIZE][MAX_ORDER_LENGTH];
int front = -1;
int rear = -1;
```

Queue declaration

To keep track of the queue's current state, the program uses two integer pointers: front and rear.

- front represents the position of the first order waiting to be served.
- rear represents the position of the most recently added order.

Initially, both pointers are set to –1, indicating that the queue is empty. When the first order is added, both pointers move to position 0. As more orders enter the system, the rear pointer increases. When an order is removed from the front, the front pointer moves forward. If all orders have been processed, both pointers reset to –1, restoring the queue to its original empty state. This mechanism ensures that the queue structure always accurately reflects the real sequence of the restaurant's order processing.

```
int isEmpty() {
    return (front == -1 || front > rear);
}
```

isEmpty()

This function checks whether the queue has any items. It returns true when:

- front is –1 (no orders ever added), or
- front has crossed rear (all orders removed).

This prevents invalid operations such as removing from or listing an empty queue and avoids a queue "underflow."

```
int isFull() {
    return (rear == MAX_SIZE - 1);
}
```

isFull()

This function verifies whether the queue has reached its maximum capacity. Since the queue uses a fixed-size array, it becomes full when rear reaches MAX_SIZE - 1. This check prevents adding more orders than the array can store and avoids "overflow."

```
void addOrder() {
    clearScreen();
    printf("\n=========================================\n");
    printf("           ADD ORDER                    \n");
    printf("=========================================\n\n");

    char orderName[MAX_ORDER_LENGTH];

    printf("Enter order name: ");
    fgets(orderName, MAX_ORDER_LENGTH, stdin);
    orderName[strcspn(orderName, "\n")] = 0;

    if(strlen(orderName) == 0) {
        printf("\nOrder name cannot be empty!\n");
        pauseScreen();
        return;
    }

    if(isFull()) {
        printf("\nQueue is full! Cannot add more orders.\n");
        pauseScreen();
        return;
    }

    if(front == -1) {
        front = 0;
    }
    rear++;
    strcpy(queue[rear], orderName);
```

addOrder() → enqueue operation

This function adds a new food order to the queue. Before adding, it checks for:

- empty queue (initially sets front to 0)
- full queue (stops adding when at capacity)

If everything is valid, rear is increased and the new order is stored at that position. This simulates a customer placing a new order at the end of the line.

```
void removeOrder() {
    clearScreen();
    printf("\n=========================================\n");
    printf("         REMOVE ORDER                    \n");
    printf("=========================================\n\n");

    if(isEmpty()) {
        printf("No orders to remove.\n");
        printf("Queue is empty!\n");
        pauseScreen();
        return;
    }

    printf("Removing order: %s\n", queue[front]);
    front++;

    if(front > rear) {
        front = -1;
        rear = -1;
        printf("\nFirst order removed successfully!\n");
        printf("Queue is now empty.\n");
    }
```

removeOrder() → dequeue operation

This function removes the oldest order from the queue. If the queue is empty, it prevents removal and displays a warning. If the order is successfully removed, front increases by one. When the last order is removed, the queue resets to its empty state by setting front and rear back to –1. This exactly represents how real kitchens complete orders one by one.

```
void listOrders() {
    clearScreen();
    printf("\n=========================================\n");
    printf("         LIST OF ORDERS                  \n");
    printf("=========================================\n\n");

    if(isEmpty()) {
        printf("No orders in queue.\n");
        printf("Queue is empty!\n");
        pauseScreen();
        return;
    }
```

listOrder() → traverse queue

This function displays all current orders from front to rear. No modifications are made to the queue; it simply reads the existing values. This representation helps staff see the order sequence without disrupting it.

```
    else {
        int total = rear - front + 1;
```

totalOrders() → size of queue

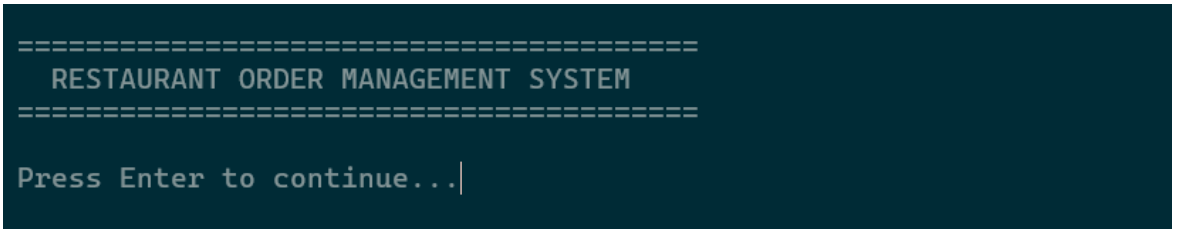
The total number of active orders is calculated using the formula : rear - front + 1

This function not only displays the count but also identifies the current first and last orders, helping staff

understand how busy the kitchen is at a glance.

3.3 Output explanation

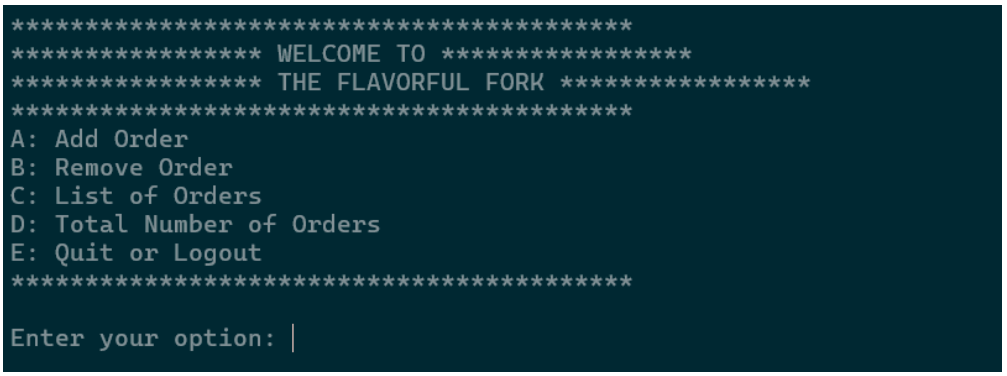
When the program is executed, the first output displayed on the screen is the system title :



When the program first runs, it displays a title screen showing that the Restaurant Order Management System has launched. This output is generated by the main() function and serves as an introduction to the system.This introduction gives users a clear indication that the application is active and ready. After pressing Enter, the user is taken to the main menu.

Main menu output

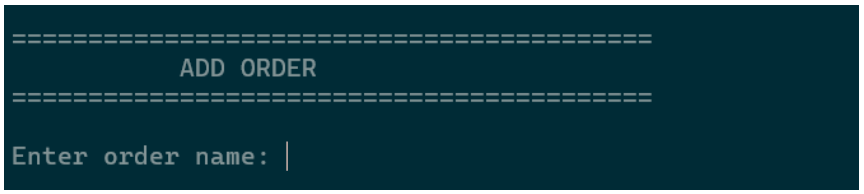
Each time the user returns to the main menu, the following message appears :



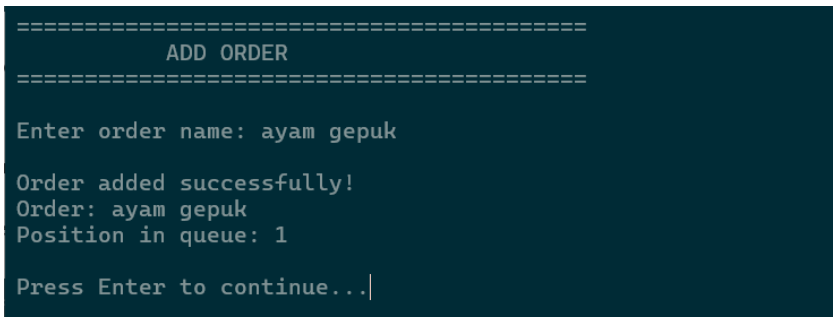
This menu is generated by displayMainMenu() and helps guide the user to the correct operation. The menu appears repeatedly after every action, making the system feel smooth and user-friendly.

Output for option A (Add Order)

When the user selects A, the program displays :



When the user types an order name, the program confirms :



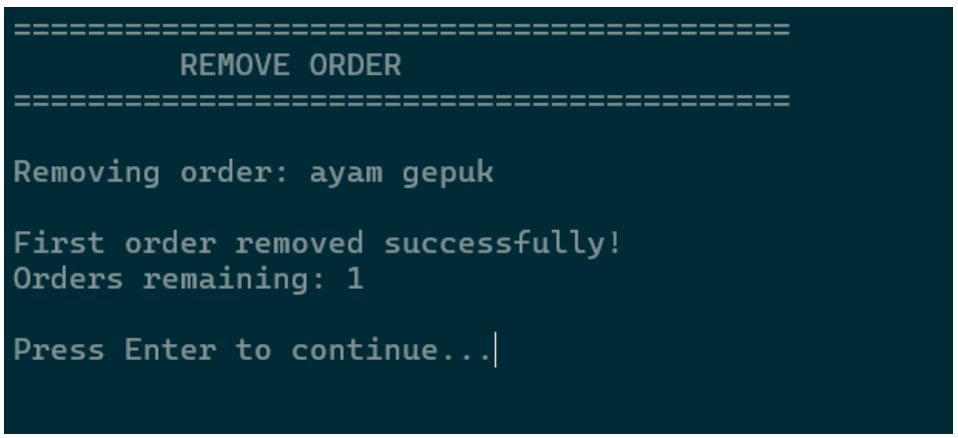
How this relates to queue processing :

This confirms that the enqueue process worked successfully .It inserts the new order at the rear of the queue. If the queue is full or if the user enters an empty name, meaningful error messages are shown. This mimics the action of adding a customer's order to the waiting list.

Output for option B (Remove Order)

If the user selects B, the program attempts to remove the earliest order in the queue. The output depends on the queue's condition:

If the queue has orders, the program prints the name of the order being removed and updates the number of orders remaining.



If removing the last order, the system informs the user that the queue is now empty.

```
Queue is now empty.
```

If no orders exist at all, a message such as “No orders to remove” is displayed.

```
No orders to remove.
Queue is empty!
```

This output clearly reflects FIFO behavior: the first order added is always the first removed.

Output for option C (List Orders)

When the user selects C, the screen shows :

```
==========================================
        LIST OF ORDERS
==========================================

Current Orders in Queue:
------------------------------------------
No.    Order Name
------------------------------------------
1      ayam gepuk
2      bakso
3      nasi lemak
------------------------------------------
Total: 3 order(s) in queue

Press Enter to continue...
```

If the queue is empty, a message stating “No orders in queue” is shown instead.

```
==========================================
        LIST OF ORDERS
==========================================

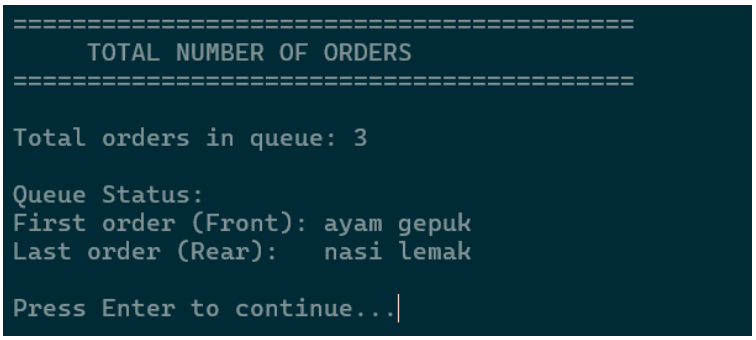
No orders in queue.
Queue is empty!

Press Enter to continue...
```

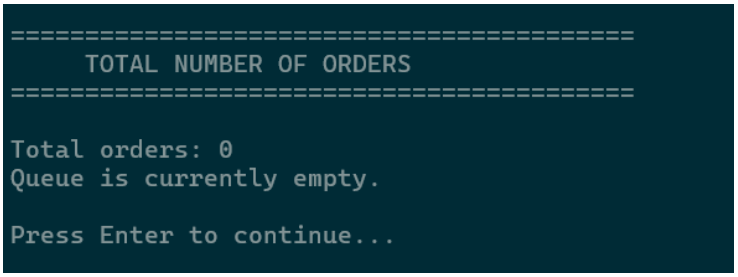
This operation does not alter the queue. It simply traverses from front to rear and prints the current sequence, demonstrating how queue traversal works.

Output for option D (Total Number of Orders)

When the user choose D, the system calculates and displays:



If empty :



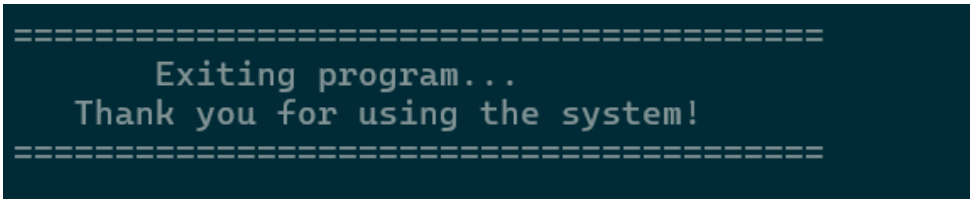
The program simply displays “0 orders.” This output helps staff understand the workload and monitor kitchen performance.

Queue relation :

- Shows the queue size, computed using rear - front + 1.
- Displays the boundaries of the queue (front and rear elements).
- Helps the user understand the current queue structure.

Output for option E (Quit/Logout)

When the user selects E, the system prints :



Selecting E clears the screen and shows an exit message confirming that the program is closing. It then terminates using exit(0) and returns the user to the normal terminal. This ensures a clean and proper shutdown without leaving stray memory or processes running.

4.0 Conclusion

In conclusion, the Cooking Order Queue System created for "The Flavorful Fork" demonstrates how an essential data structure like the Queue can be applied effectively to real-life situations. By using the FIFO (First-In, First-Out) principle, the system ensures that food orders are processed in the exact sequence they are received, which is critical for maintaining fairness, accuracy, and smooth workflow inside a busy kitchen environment. This project successfully shows how queue operations—such as adding new orders, removing completed ones, displaying the list of current orders, and counting pending orders—can be used to manage tasks efficiently and systematically.

From an academic perspective, the development of this system provides meaningful learning opportunities. It strengthens students' understanding of fundamental data structures and enhances their skills in algorithm design, logical decision-making, and modular programming in C. By translating real restaurant activities into program logic, the project helps students gain practical experience in applying theoretical concepts to functional software. This kind of hands-on practice is valuable for building confidence and competence in problem-solving and system development.

Even though the queue method is simple and well-suited for sequential processing, it does come with limitations. For example, queues do not allow easy direct access to the middle or last elements, and static queue implementations may encounter overflow issues if the array size is fixed. Despite these limitations, the queue remains a highly appropriate and effective structure for the scope of this project because restaurant orders naturally follow a strict sequence.

Overall, the project achieves its intended objectives by providing a clear and efficient solution for managing cooking orders. It also reinforces the importance of choosing the correct data structure when designing software systems. The Cooking Order Queue System is not only practical for the given scenario but also aligns well with the learning outcomes of GMI's Data Structure and Algorithms course, helping students prepare for more complex programming challenges in future modules.

5.0 References

- Safa Mulani. (2022, August 3). *How to Create a Queue in C (With Code Examples)*. Digitalocean.com; DigitalOcean. https://www.digitalocean.com/community/tutorials/queue-in-c%0A
- GeeksforGeeks. (2024, May 6). *Queue in C*. GeeksforGeeks. https://www.geeksforgeeks.org/c/queue-in-c/%0A
- *W3Schools.com*. (2025). W3schools.com. https://www.w3schools.com/cpp/cpp_queues.asp%0
- NG COMPUTER. (2025). *Food Ordering System in C*. Scribd. https://www.scribd.com/document/699054072/Food-Ordering-System-in-C